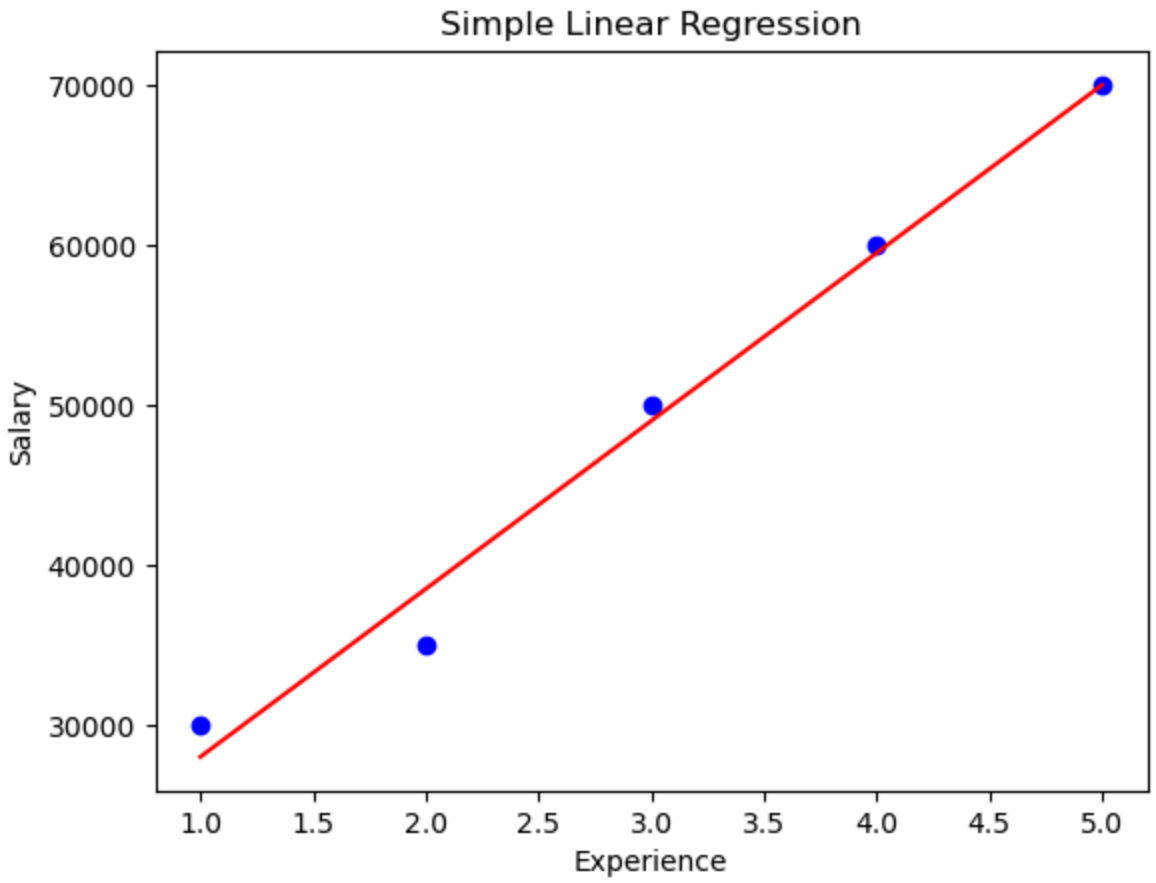
Chapter-4 Regression –Model Training and Evaluation

In [ ]:
Linear regression models the relationship between:
• Independent variable(s) (X)
• Dependent variable (y)
It fits a line (or hyperplane) to predict values.

Simple Linear Regression (One Feature) (y = mx + c)

Example 1: Predict salary based on years of experience

In [1]:
```
# Step 1: Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Step 2: Create dataset
# Independent variable (Years of Experience)
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)

# Dependent variable (Salary)
y = np.array([30000, 35000, 50000, 60000, 70000])
# Step 3: Train model
model = LinearRegression()
model.fit(x, y)
# Step 4: Predictions
y_pred = model.predict(x)
# Step 5: Visualization
plt.scatter(x, y, color='blue')      # actual data
plt.plot(x, y_pred, color='red')     # regression line
plt.xlabel("Experience")
plt.ylabel("Salary")
plt.title("Simple Linear Regression")
plt.show()
# Step 6: Model parameters
print("Slope:", model.coef_)
print("Intercept:", model.intercept_)
```



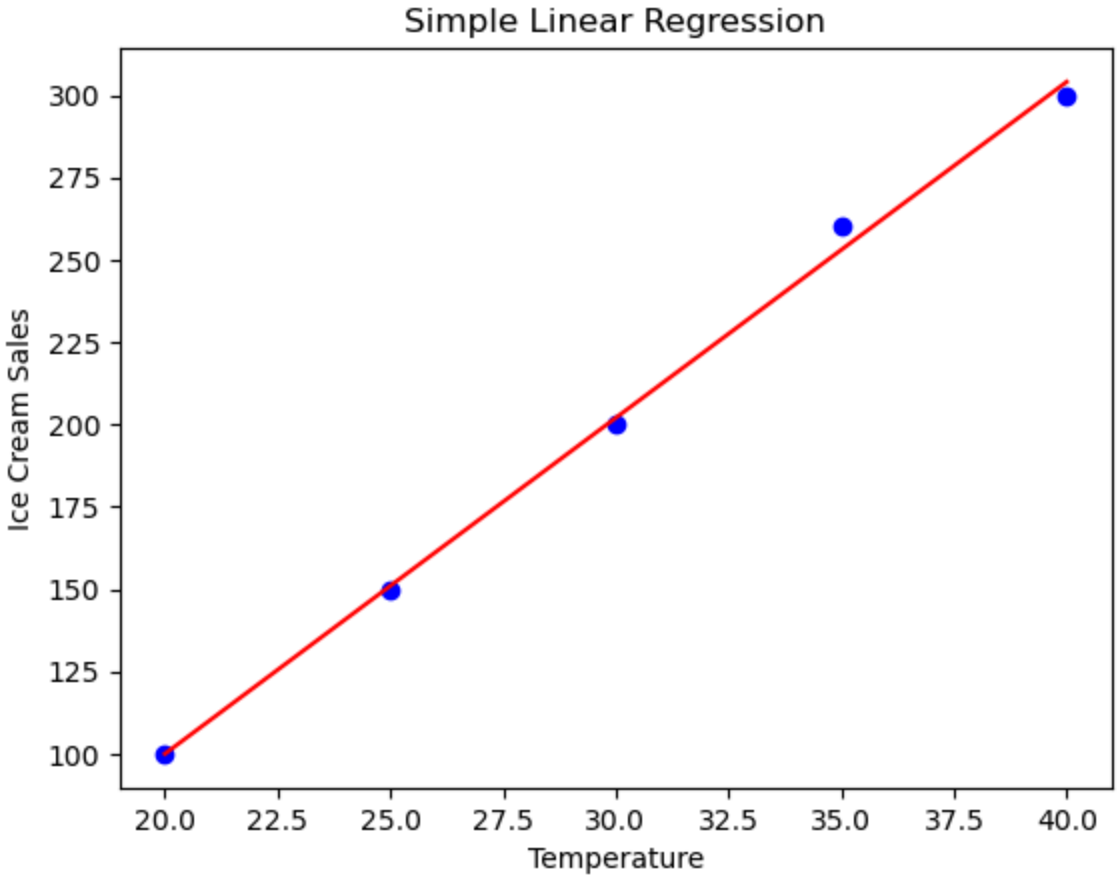

```
Slope: [10500.]
Intercept: 17499.999999999993
```

Example 2: Temperature vs Ice Cream Sales

In [2]:
```python
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Step 2: Create dataset
# Temperature (°C)
x = np.array([20, 25, 30, 35, 40]).reshape(-1, 1)

# Ice cream sales
y = np.array([100, 150, 200, 260, 300])

# Step 3: Train model
model = LinearRegression()
model.fit(x, y)

#  Step 4: Predict
# Predict sales at 32°C
predicted_sales = model.predict([[32]])
print("Predicted Sales:", predicted_sales)

# Step 5: Visualize
```

```
y_pred = model.predict(x)

plt.scatter(x, y, color='blue')     # actual data
plt.plot(x, y_pred, color='red')    # regression line
plt.xlabel("Temperature")
plt.ylabel("Ice Cream Sales")
plt.title("Simple Linear Regression")
plt.show()

# Step 6: Understand model
print("Slope (m):", model.coef_)
print("Intercept (c):", model.intercept_)
```

```
Predicted Sales: [222.4]
```




```
Slope (m): [10.2]
Intercept (c): -104.0
```

Multiple Linear Regression (Multiple Features) ($y = b_0 + b_1x_1 + b_2x_2 + \dots$)

Example 1: Predict house price using size & number of rooms

In [3]:
```
# Step 1: Dataset
# Features: [Size (sqft), Number of Rooms]
x = np.array([
    [1000, 2],
    [1500, 3],
    [2000, 4],
    [2500, 4],
    [3000, 5]
])

# Target: Price
y = np.array([200000, 300000, 400000, 450000, 500000])
# Step 2: Train model
model = LinearRegression()
model.fit(x, y)
# Step 3: Prediction
# Predict price for a house of 2200 sqft with 3 rooms
prediction = model.predict([[2200, 3]])
print("Predicted Price:", prediction)
# Step 4: Coefficients
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Predicted Price: [356000.]
Coefficients: [   80. 50000.]
Intercept: 30000.0
```

Example 2:House Price Prediction

In [4]:
```
# Step 1: Import libraries
import numpy as np
from sklearn.linear_model import LinearRegression

# Step 2: Create dataset
# Features: [Area, Rooms, Age, Location Score]
x = np.array([
    [1000, 2, 10, 3],
    [1500, 3, 5, 4],
    [2000, 4, 2, 5],
    [2500, 4, 1, 5],
    [3000, 5, 1, 5]
])

# House prices
y = np.array([5000000, 8000000, 12000000, 15000000, 18000000])

# Step 3: Train model
model = LinearRegression()
model.fit(x, y)

# Step 4: Predict
# Predict price for:
# Area=2200, Rooms=3, Age=3, Location Score=4
prediction = model.predict([[2200, 3, 3, 4]])
```

```
print("Predicted Price:", prediction)

#  Step 5: Understand coefficients
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Predicted Price: [11900000.]
Coefficients: [   7000. -500000.  500000. 2500000.]
Intercept: -13500000.00000003
```

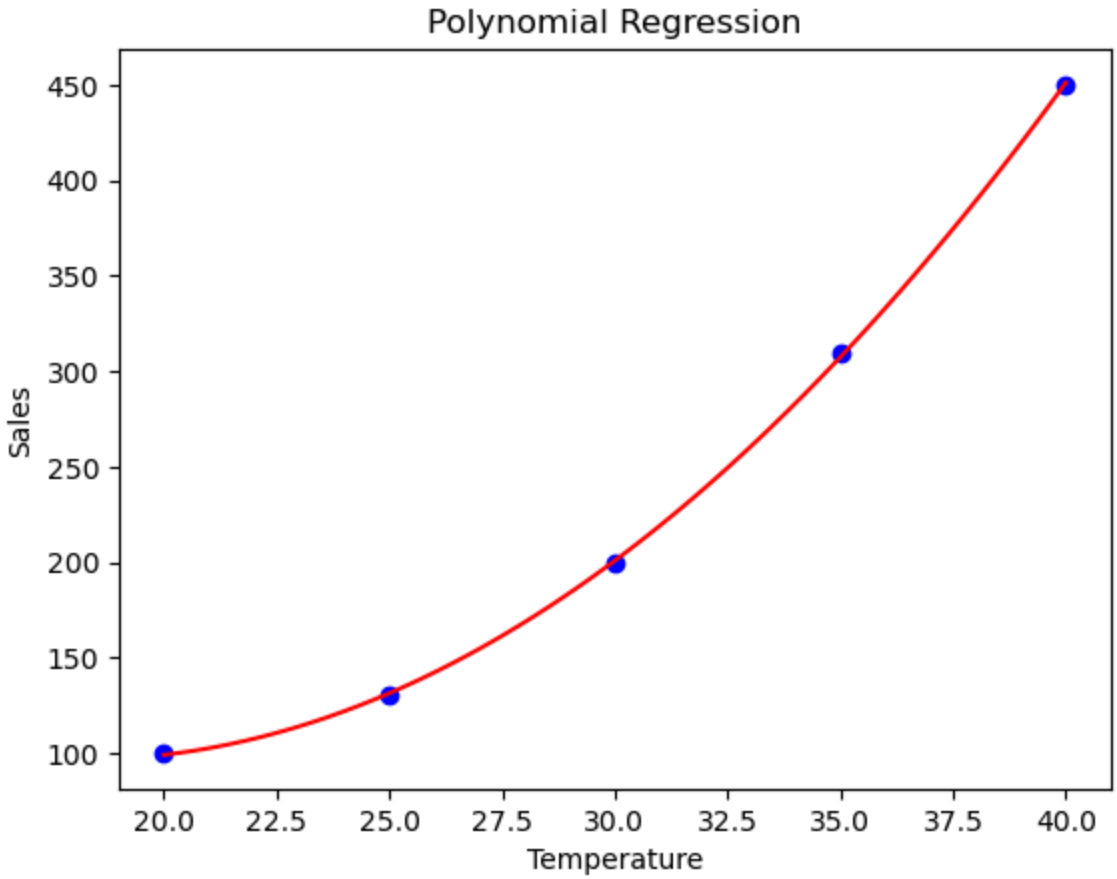
Polynomial Regression

In [ ]:
```
It's an extension of linear regression where:
• Instead of fitting a straight line, we fit a curve
• Useful when data shows a non-linear relationship
     Example: Ice cream sales may increase slowly, then rapidly as temperature rise
```

In [5]:
```
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Create Dataset (Non-linear)
# Temperature
x = np.array([20, 25, 30, 35, 40]).reshape(-1, 1)

# Sales (non-linear pattern)
y = np.array([100, 130, 200, 310, 450])

# Convert to Polynomial Features
poly = PolynomialFeatures(degree=2)   # degree 2 = quadratic
x_poly = poly.fit_transform(x)

# Train Model
model = LinearRegression()
model.fit(x_poly, y)


# Predict for 32°C
prediction = model.predict(poly.transform([[32]]))
print("Predicted Sales:", prediction)

# Visualization (Curve instead of line)
# Smooth curve
x_range = np.linspace(20, 40, 100).reshape(-1, 1)
x_range_poly = poly.transform(x_range)
y_range = model.predict(x_range_poly)

plt.scatter(x, y, color='blue')        # actual points
plt.plot(x_range, y_range, color='red')  # curve
plt.xlabel("Temperature")
plt.ylabel("Sales")
```

```
plt.title("Polynomial Regression")
plt.show()

# Model Equation
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Predicted Sales: [239.02857143]
```



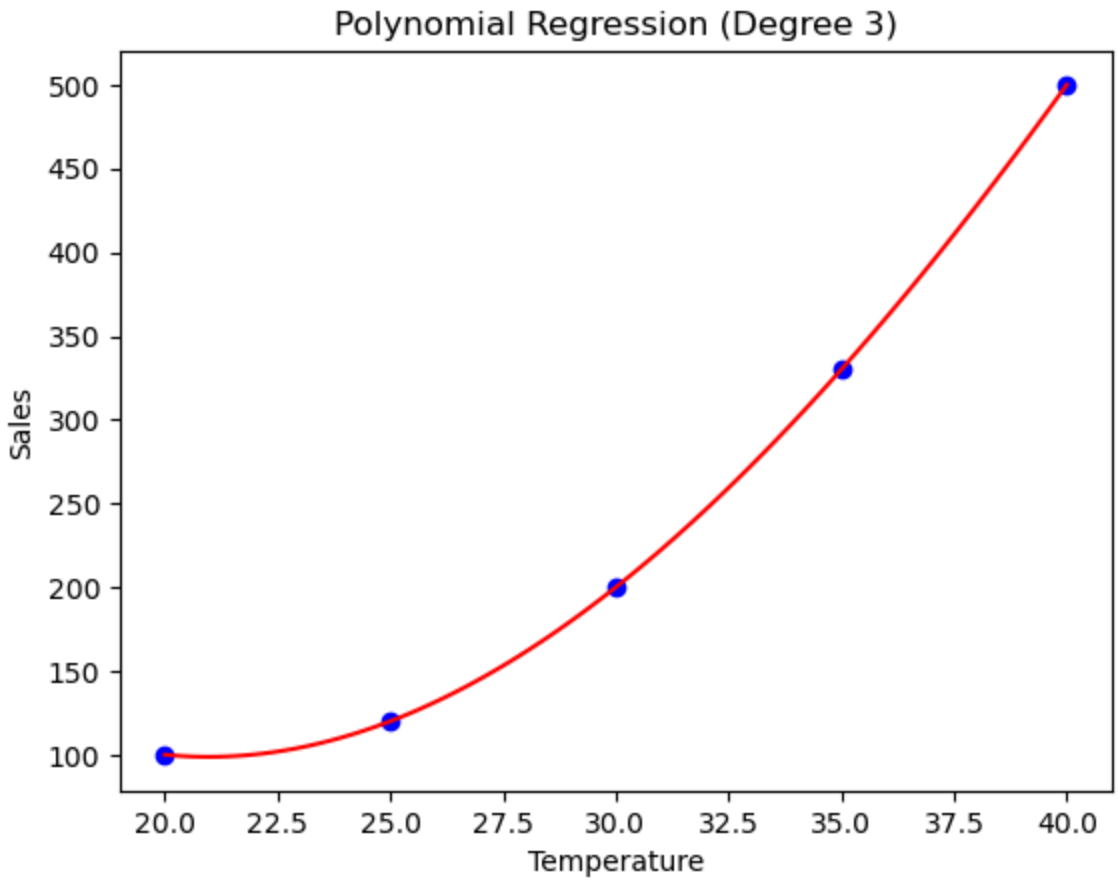

```
Coefficients: [  0.         -26.97142857   0.74285714]
Intercept: 341.42857142857974
```

Polynomial Regression (Degree = 3)

In [6]:
```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

#  2. Dataset (Non-Linear)
# Temperature
x = np.array([20, 25, 30, 35, 40]).reshape(-1, 1)

# Sales (more curved pattern)
y = np.array([100, 120, 200, 330, 500])

#  3. Create Polynomial Features (Degree 3)
```

```
poly = PolynomialFeatures(degree=3)
x_poly = poly.fit_transform(x)

# Train Model
model = LinearRegression()
model.fit(x_poly, y)

# Predict Value
# Predict sales at 32°C
prediction = model.predict(poly.transform([[32]]))
print("Predicted Sales:", prediction)

#  Plot the Curve
# Smooth curve for better visualization
x_range = np.linspace(20, 40, 100).reshape(-1, 1)
x_range_poly = poly.transform(x_range)
y_range = model.predict(x_range_poly)

plt.scatter(x, y, color='blue')        # actual data
plt.plot(x_range, y_range, color='red') # cubic curve
plt.xlabel("Temperature")
plt.ylabel("Sales")
plt.title("Polynomial Regression (Degree 3)")
plt.show()

# Model Equation
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Predicted Sales: [246.56]
```

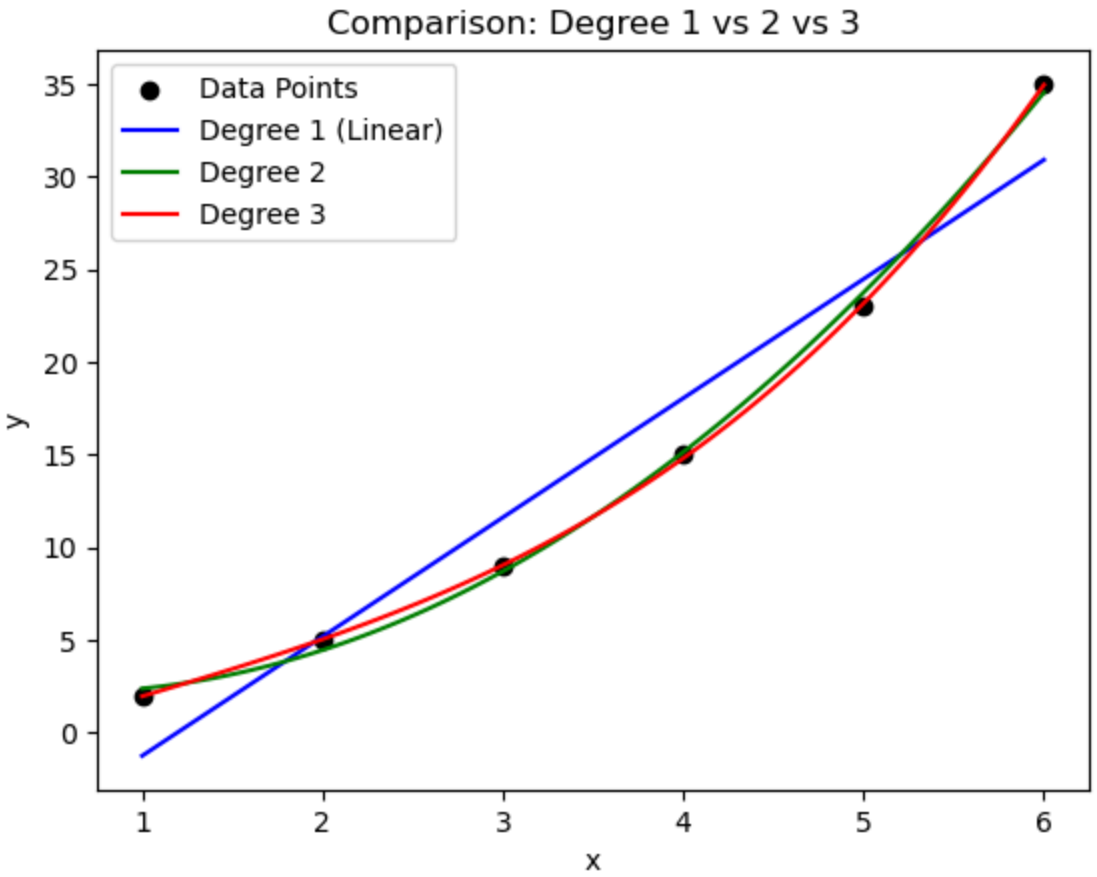



```
Coefficients: [ 0.00000000e+00 -7.46666667e+01  2.20000000e+00 -1.33333333e-02]
Intercept: 819.9999999938182
```

Compare Degree 1 vs 2 vs 3 (One Graph)

In [7]:
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Dataset (non-linear)
x = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([2, 5, 9, 15, 23, 35])

# Smooth values for curve plotting
x_range = np.linspace(1, 6, 100).reshape(-1, 1)

# -------- Degree 1 (Linear) --------
lin_model = LinearRegression()
lin_model.fit(x, y)
y_lin = lin_model.predict(x_range)

# -------- Degree 2 --------
poly2 = PolynomialFeatures(degree=2)
x_poly2 = poly2.fit_transform(x)
model2 = LinearRegression()
model2.fit(x_poly2, y)
```

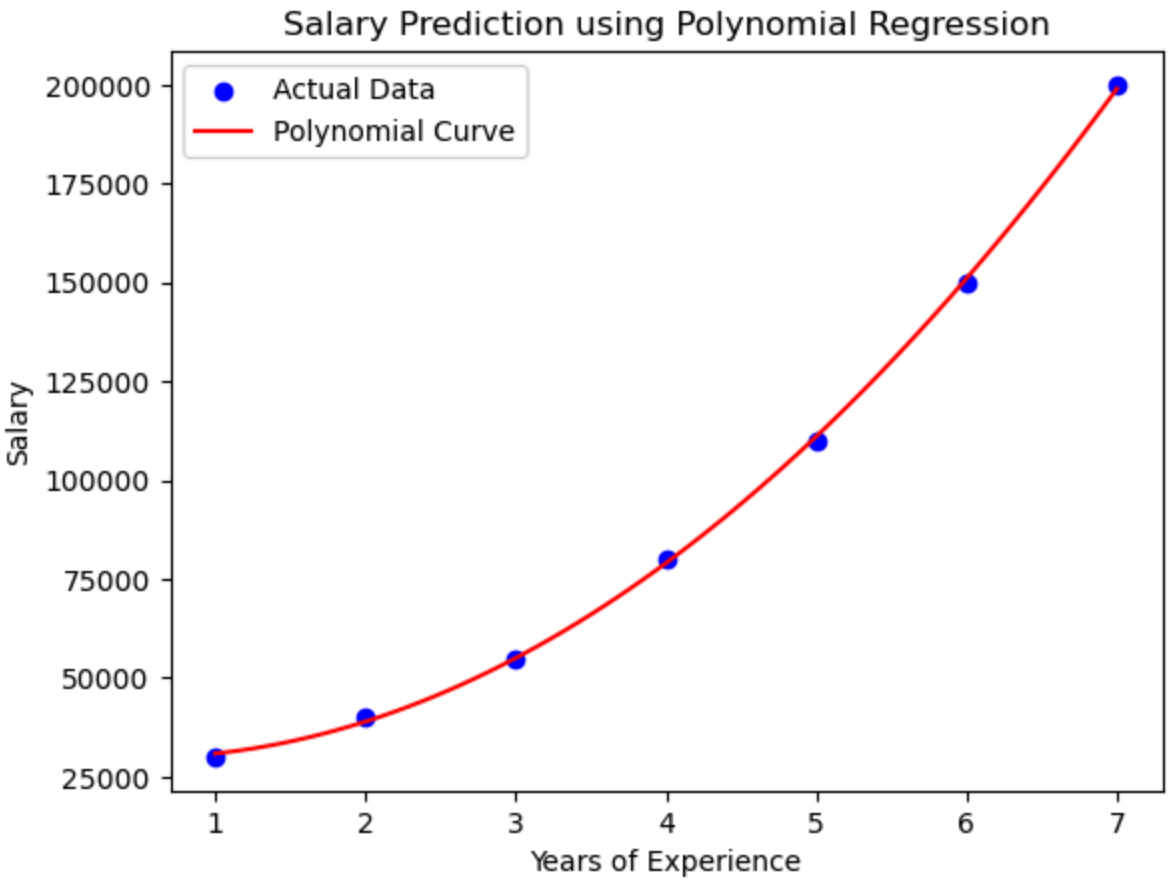
```
y_poly2 = model2.predict(poly2.transform(x_range))

# -------- Degree 3 --------
poly3 = PolynomialFeatures(degree=3)
x_poly3 = poly3.fit_transform(x)
model3 = LinearRegression()
model3.fit(x_poly3, y)
y_poly3 = model3.predict(poly3.transform(x_range))

# -------- Plot all --------
plt.scatter(x, y, color='black', label='Data Points')

plt.plot(x_range, y_lin, color='blue', label='Degree 1 (Linear)')
plt.plot(x_range, y_poly2, color='green', label='Degree 2')
plt.plot(x_range, y_poly3, color='red', label='Degree 3')

plt.xlabel("x")
plt.ylabel("y")
plt.title("Comparison: Degree 1 vs 2 vs 3")
plt.legend()
plt.show()
```



Predict salary based on years of experience (non-linear growth)

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Years of Experience
x = np.array([1, 2, 3, 4, 5, 6, 7]).reshape(-1, 1)

# Salary (non-linear growth)
y = np.array([30000, 40000, 55000, 80000, 110000, 150000, 200000])

# Polynomial transformation
poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(x)

# Train model
model = LinearRegression()
model.fit(x_poly, y)

# Predict salary for 6.5 years
predicted_salary = model.predict(poly.transform([[6.5]]))
print("Predicted Salary:", predicted_salary)

x_range = np.linspace(1, 7, 100).reshape(-1, 1)
y_range = model.predict(poly.transform(x_range))

plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x_range, y_range, color='red', label='Polynomial Curve')
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Salary Prediction using Polynomial Regression")
plt.legend()
plt.show()
```

```
Predicted Salary: [174062.5]
```



Evaluation using R-squared, MAE, MSE

In [ ]:
R² Score (Coefficient of Determination)
- Measures how well model fits data
- Range: 0 to 1
- Closer to 1 = better fit

In [ ]:
MAE (Mean Absolute Error)
- Average absolute error
- Easy to interpret

In [ ]:
MSE (Mean Squared Error)
Penalizes large errors more

Linear Regression Evaluation

In [9]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
# Dataset
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([3, 4, 2, 5, 6])
# Train Model
```

```
model = LinearRegression()
model.fit(x, y)
# Predictions
y_pred = model.predict(x)
# Evaluation Metrics
r2 = r2_score(y, y_pred)
mae = mean_absolute_error(y, y_pred)
mse = mean_squared_error(y, y_pred)

print("R2 Score:", r2)
print("MAE:", mae)
print("MSE:", mse)
```

```
R2 Score: 0.49
MAE: 0.7999999999999999
MSE: 1.02
```

Polynomial Regression Evaluation

In [10]:
```
from sklearn.preprocessing import PolynomialFeatures

# Transform to degree 2
poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(x)

model = LinearRegression()
model.fit(x_poly, y)

y_pred_poly = model.predict(x_poly)

# Metrics
print("Polynomial R2:", r2_score(y, y_pred_poly))
print("Polynomial MAE:", mean_absolute_error(y, y_pred_poly))
print("Polynomial MSE:", mean_squared_error(y, y_pred_poly))
```

```
Polynomial R2: 0.6685714285714285
Polynomial MAE: 0.6857142857142846
Polynomial MSE: 0.662857142857143
```

In [ ]:
```
# Programs using CSV
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

# -------------------------------
# 1. Load Dataset from CSV
# -------------------------------
data = pd.read_csv("salary_data.csv")

x = data[['YearsExperience']]   # independent variable
```

```
y = data['Salary']              # dependent variable
# -------------------------------
# 2. Train-Test Split
# -------------------------------
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=0
)

# -------------------------------
# 3. Linear Regression
# -------------------------------
lin_model = LinearRegression()
lin_model.fit(x_train, y_train)

y_pred_lin = lin_model.predict(x_test)

# -------------------------------
# 4. Polynomial Regression (Degree 2)
# -------------------------------
poly = PolynomialFeatures(degree=2)

x_train_poly = poly.fit_transform(x_train)
x_test_poly = poly.transform(x_test)

poly_model = LinearRegression()
poly_model.fit(x_train_poly, y_train)

y_pred_poly = poly_model.predict(x_test_poly)

# -------------------------------
# 5. Evaluation Function
# -------------------------------
def evaluate(y_true, y_pred, model_name):
    print(f"\n--- {model_name} ---")
    print("R2 Score:", r2_score(y_true, y_pred))
    print("MAE:", mean_absolute_error(y_true, y_pred))
    print("MSE:", mean_squared_error(y_true, y_pred))

# -------------------------------
# 6. Compare Models
# -------------------------------
evaluate(y_test, y_pred_lin, "Linear Regression")
evaluate(y_test, y_pred_poly, "Polynomial Regression")

# -------------------------------
# 7. Visualization
# -------------------------------
x_range = np.linspace(x.min(), x.max(), 100)
x_range_df = pd.DataFrame(x_range, columns=['YearsExperience'])

# Linear prediction
y_range_lin = lin_model.predict(x_range_df)

# Polynomial prediction
y_range_poly = poly_model.predict(poly.transform(x_range_df))
```

```
plt.scatter(x, y, color='black', label='Actual Data')
plt.plot(x_range, y_range_lin, color='blue', label='Linear')
plt.plot(x_range, y_range_poly, color='red', label='Polynomial (Degree 2)')

plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Regression using CSV Dataset")
plt.legend()
plt.show()
```

In [ ]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

# --------------------------------
# 1. Load CSV Dataset
# --------------------------------
data = pd.read_csv("house_data.csv")

# Features and Target
x = data[['Area', 'Rooms', 'Age', 'Location']]
y = data['Price']
# --------------------------------
# 2. Train-Test Split
# --------------------------------
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=0
)

# --------------------------------
# 3. Multiple Linear Regression
# --------------------------------
lin_model = LinearRegression()
lin_model.fit(x_train, y_train)

y_pred_lin = lin_model.predict(x_test)

# --------------------------------
# 4. Polynomial Regression (Degree 2)
# --------------------------------
poly = PolynomialFeatures(degree=2)

x_train_poly = poly.fit_transform(x_train)
x_test_poly = poly.transform(x_test)

poly_model = LinearRegression()
poly_model.fit(x_train_poly, y_train)

y_pred_poly = poly_model.predict(x_test_poly)

# --------------------------------
```

```
# 5. Evaluation Function
# ------------------------------
def evaluate(y_true, y_pred, model_name):
    print(f"\n--- {model_name} ---")
    print("R2 Score:", r2_score(y_true, y_pred))
    print("MAE:", mean_absolute_error(y_true, y_pred))
    print("MSE:", mean_squared_error(y_true, y_pred))

# ------------------------------
# 6. Compare Models
# ------------------------------
evaluate(y_test, y_pred_lin, "Multiple Linear Regression")
evaluate(y_test, y_pred_poly, "Polynomial Regression")
# ------------------------------
# 7. Predict New House Price
# ------------------------------
# Example: Area=2200, Rooms=3, Age=3, Location=2
new_house = [[2200, 3, 3, 2]]
lin_price = lin_model.predict(new_house)
poly_price = poly_model.predict(poly.transform(new_house))

print("\nPredicted Price (Linear):", lin_price)
print("Predicted Price (Polynomial):", poly_price)
```

In [ ]:

```
# 229. Consider variables x and y created from a pandas dataframe  “car.csv” .
Create new column named “Age_car” (Age_car=2023-year)
 For multiple linear regression problem, x contains the independent variables
( Age_car , Driven_kms , Fuel_Type , Selling_type , Transmission )  and y contains
(Selling_Price) variable which is to be predicted.
 Write a Python program to spilt x and y into training and testing datasets with a
Then create a multiple linear regression model using the training data and print it
```

In [ ]:

```
# =========================
# Import Required Libraries
# =========================
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error


# =========================
# Load Dataset
# =========================
df = pd.read_csv("car.csv")

print("First 5 rows:")
print(df.head())


# =========================
# Create New Column: Age_car
# =========================
```

```
df['Age_car'] = 2023 - df['Year']


# =========================
# Select Features (X) and Target (y)
# =========================
X = df[['Age_car', 'Driven_kms', 'Fuel_Type', 'Selling_type', 'Transmission']]
y = df['Selling_Price']


# =========================
# Convert Categorical Data
# =========================
# Convert text columns into numeric using one-hot encoding
X = pd.get_dummies(X, drop_first=True)


# =========================
# Split Data (80% train, 20% test)
# =========================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# =========================
# Train Multiple Linear Regression Model
# =========================
model = LinearRegression()
model.fit(X_train, y_train)


# =========================
# Predictions
# =========================
y_pred = model.predict(X_test)


# =========================
# Output Results
# =========================
print("\nIntercept:")
print(model.intercept_)

print("\nCoefficients:")
for feature, coef in zip(X.columns, model.coef_):
    print(f"{feature}: {coef}")

print("\nMean Squared Error (MSE):")
print(mean_squared_error(y_test, y_pred))
```

In [ ]:

```
# 230. Write a python program for following tasks -
Make sure all the required libraries are imported at the beginning of the Jupyter N
a) Load the windpower dataset into the notebook.
b) Check records for any missing values and remove them permanently using appropria
c) Treat windspeed as the independent variable and power as the dependent variable.
```

```
    Split the dataset into training (75%) and testing (25%) sets using random state
d) Use scikit-learn to train polynomial regression models of degrees 3, 4, 5, and 6
e) Predict power values on the test set and print the Mean Squared Error (MSE) and
(coefficient of determination) for each polynomial degree.
f) Plot actual vs. predicted power values for all polynomial degrees on the same gr
    for comparison and label the plot accordingly.
g) Create a plot showing MSE vs. polynomial degree. Which degree has the lowest MSE
h) Print the coefficients and intercept for each trained polynomial regression mode
i) Use each model to predict power output for wind speeds of 5, 10, 15, and 20 m/s,
```

In [ ]:

```
# =========================
# Import Required Libraries
# =========================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score


# =========================
# a) Load Dataset
# =========================
# Make sure windpower.csv is in your working directory
df = pd.read_csv("windpower.csv")

print("First 5 rows:")
print(df.head())


# =========================
# b) Handle Missing Values
# =========================
print("\nMissing values before cleaning:")
print(df.isnull().sum())

# Remove missing values permanently
df = df.dropna()

print("\nMissing values after cleaning:")
print(df.isnull().sum())


# =========================
# c) Define Variables & Split
# =========================
X = df[['windspeed']]   # Independent variable
y = df['power']         # Dependent variable

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
# ========================
# d) Train Polynomial Models
# ========================
degrees = [3, 4, 5, 6]
models = {}
mse_list = []

plt.figure(figsize=(10, 6))

# ========================
# e) Train, Predict, Evaluate
# ========================
for degree in degrees:
    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    mse_list.append(mse)
    models[degree] = (model, poly)

    print(f"\nDegree {degree}:")
    print("MSE:", mse)
    print("R2 Score:", r2)

    # f) Plot Actual vs Predicted
    plt.scatter(X_test, y_pred, label=f'Degree {degree}')

# Plot actual values
plt.scatter(X_test, y_test, color='black', label='Actual', alpha=0.6)

plt.xlabel("Wind Speed")
plt.ylabel("Power")
plt.title("Actual vs Predicted Power (All Degrees)")
plt.legend()
plt.show()


# ========================
# g) MSE vs Degree Plot
# ========================
plt.figure()
plt.plot(degrees, mse_list, marker='o')
plt.xlabel("Polynomial Degree")
plt.ylabel("MSE")
plt.title("MSE vs Polynomial Degree")
plt.show()
```

```
# Lowest MSE
best_degree = degrees[np.argmin(mse_list)]
print("\nBest Degree (Lowest MSE):", best_degree)

# Comment:
# The polynomial degree with the lowest MSE is considered the best model.


# ========================
# h) Print Coefficients
# ========================
for degree in degrees:
    model, poly = models[degree]
    print(f"\nDegree {degree} Coefficients:")
    print("Intercept:", model.intercept_)
    print("Coefficients:", model.coef_)


# ========================
# i) Predict New Values
# ========================
new_wind_speeds = np.array([[5], [10], [15], [20]])

print("\nPredictions for new wind speeds:")

for degree in degrees:
    model, poly = models[degree]
    new_poly = poly.transform(new_wind_speeds)
    predictions = model.predict(new_poly)

    print(f"\nDegree {degree}:")
    for ws, pred in zip(new_wind_speeds.flatten(), predictions):
        print(f"Wind Speed = {ws} m/s -> Predicted Power = {pred}")
```